

Introduction to Computer Architecture

Chapter 4 - 1

The Processor

Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

Introduction

- **Micro-architecture** means the **internal** structure of a CPU
 - ❖ Externally, the CPUs of the same ISA has the same interface.
 - ❖ Internally, the CPUs can be composed different to each other, depending on their performance/efficiency targets
- Microarchitecture examples we will study
 - ❖ **Single-cycle** *CPI=1*
 - Each instruction is executed in one cycle
 - It suffers from the long critical path delay, which limits the clock frequency *low performance, simple*
 - ❖ **Pipelined**
 - Execution is broken up into a series of steps *execution instructions (workload) ⇒ several stages*
 - Multiple instructions are executed in an overlapped fashion.
parallel *Hazard 321*

Introduction

- Our example supports only a subset of RISC-V ISA
 - ❖ Arithmetic/logic: **add**, **sub**, **and**, **or**, *xor*
 - ❖ Memory access: **lw**, **sw**, *lh*
 - ❖ Control transfer: **beq**, *bne*
branch^u equal
- Covers most common operations of a typical RISC-style CPU

What Happens in CPU

- Load an instruction from (instruction) memory (*fetch*)
 - ❖ Memory address: PC
Program Counter
- Use the opcode field to identify the instruction type (*decode*)
 - ❖ Instruction[6:0] (i.e., lower 7 bits)
- These first two steps are identical for all instruction types.

What Happens in CPU

- Arithmetic/Logic instructions: **add**, **sub**, **and**, **or**
 - ❖ Read two operand values from the register file (*rs1*, *rs2*)
 - ❖ Perform calculation → ALU
 - add
 - sub
 - and
 - or
 - ...
 - ❖ Write the result back into the register file (*rd*)
 - ❖ Add 4 to PC
 - current instruction →
 - next instruction → +4

What Happens in CPU

- Memory access: **lw** *rd, offset(rs1)*
 - ❖ Read 1 word from the register file (*rs1*): base address
 - ❖ Add the base address with the offset (*Immediate field*)
 - ❖ Load word from data memory (Address: base address $\oplus$ offset)
add
 - ❖ Write loaded word into the register file (*rd*)
 - ❖ Add 4 to PC

What Happens in CPU

- Memory access: **sw** *rs2, offset(rs1)*
 - ❖ Read 1 word from the register file (*rs1*): base address
 - ❖ Add the base address with the offset (*Immediate field*)
 - ❖ Read 1 word from the register file (*rs2*): value to store
 - ❖ Store the word to data memory (Address: base address + offset)
 - ❖ Add 4 to PC
- Handwritten notes:*
- A red arrow points from *offset(rs1)* to the "Add the base address with the offset" step.
 - A red bracket groups the first three steps, with a note: *스레 상한 X 동시해도 안 dependency X*
 - A red arrow points from value to store to the "Store the word to data memory" step.

What Happens in CPU

■ Control transfer: **beq**

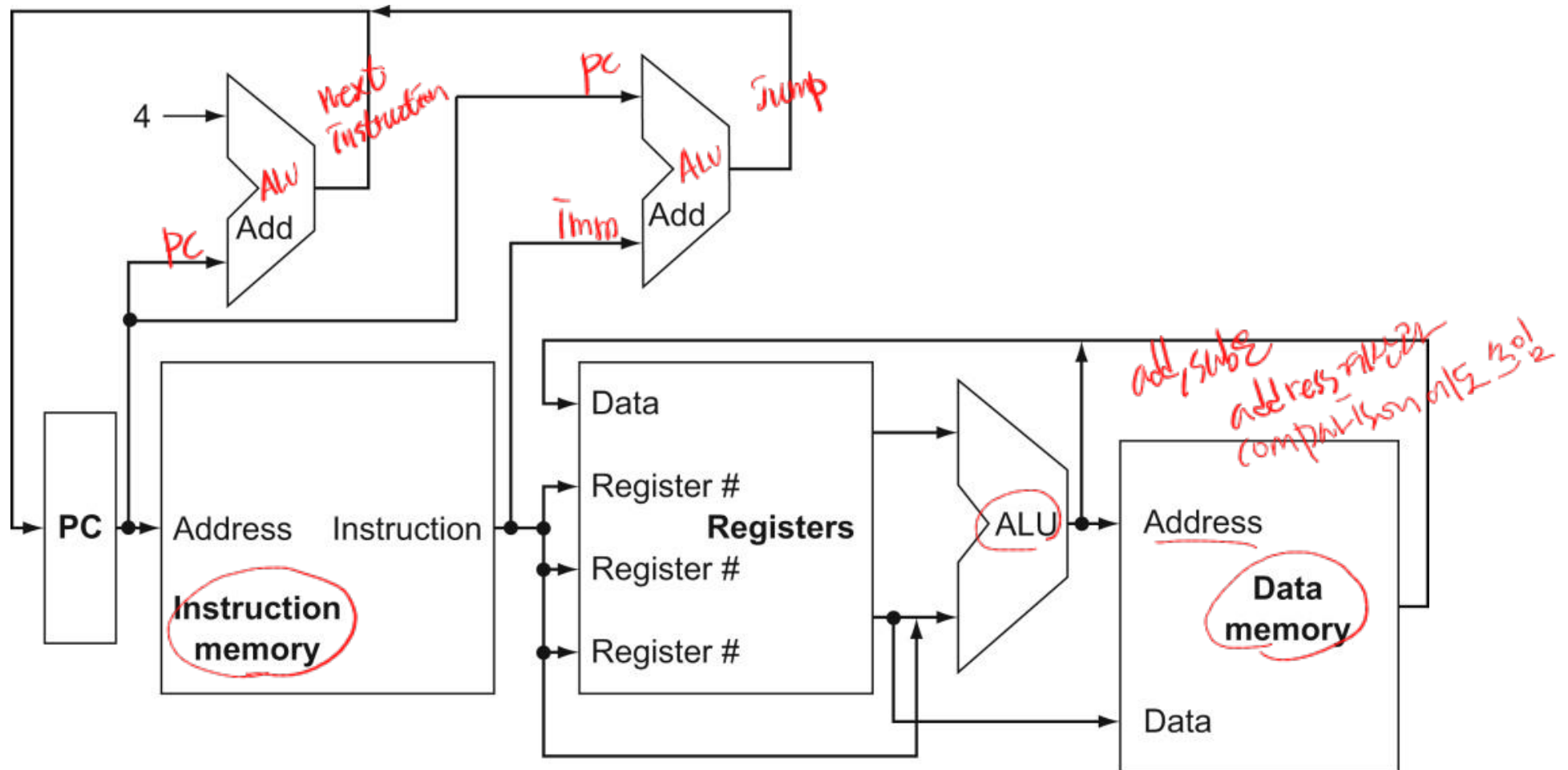
- ❖ Read 2 words from the register file ($rs1$, $rs2$)
- ❖ Perform comparison (using ALU): Subtraction ^{4B}
- ❖ If $*rs1 == *rs2$
 - $PC \leftarrow PC + \text{immediate}$
- ❖ else
 - $PC \leftarrow PC + 4$

↳ $A == B?$ $A - B = 0$



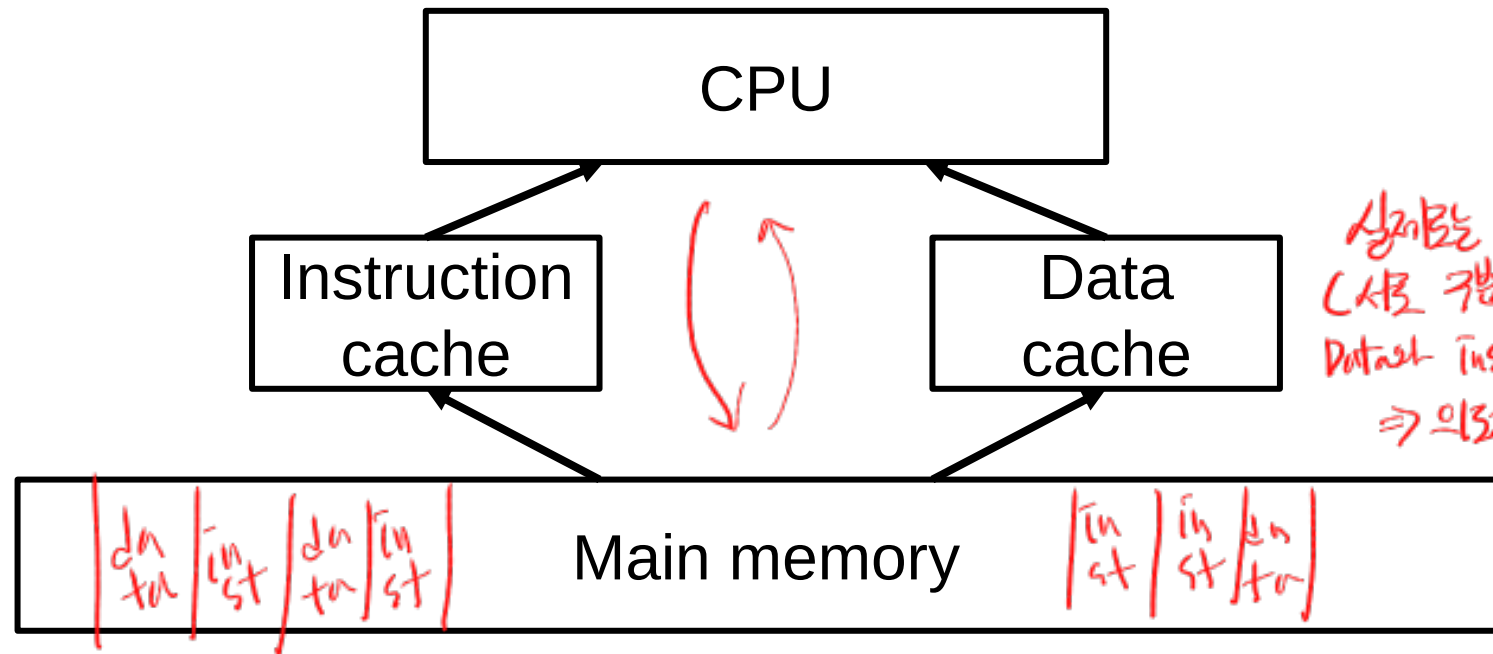
Single-cycle CPU Overview

- 1 cycle for all instructions (CPI = 1)



Instruction / Data Memory?

- When describing a processor architecture in this lecture, we use separate units for instruction memory / data memory
- In reality, these are the same main memory.
- From the processor side, they may look like separate memory because we will use separate cache memory



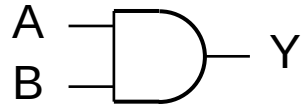
실제로는 하나의 main memory만 존재
(서로 구분하지 않음) 그러나 CPU 입장에서는
Data와 Instruction이 서로 다른 cache에 있음
⇒ 의식적으로 구분

Logic Design Basics: Combinational Logic

- Output is a function of the **current** input

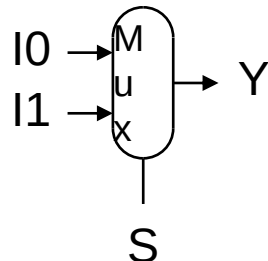
- AND-gate

- ❖ $Y = A \& B$



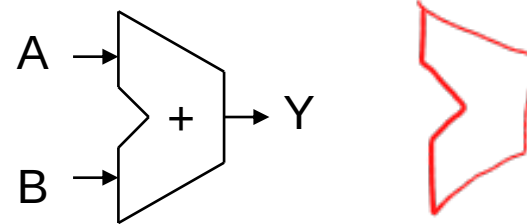
- Multiplexer

- ❖ $Y = S ? I1 : I0$



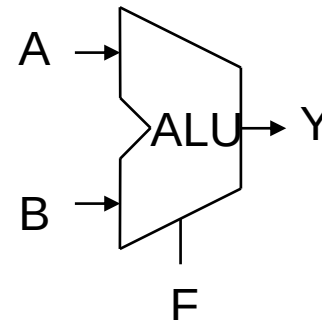
- Adder

- ❖ $Y = A + B$

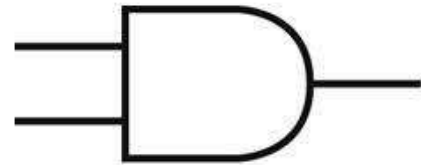


- Arithmetic/Logic Unit

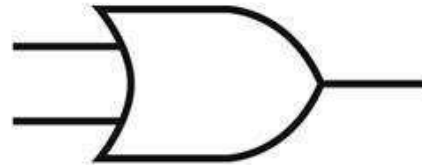
- ❖ $Y = F(A, B)$



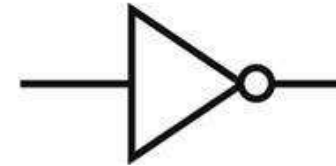
Logic Design Basics: Logic Gates



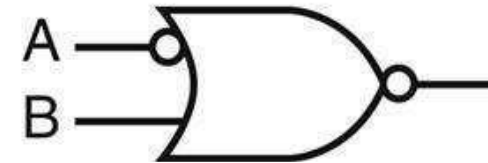
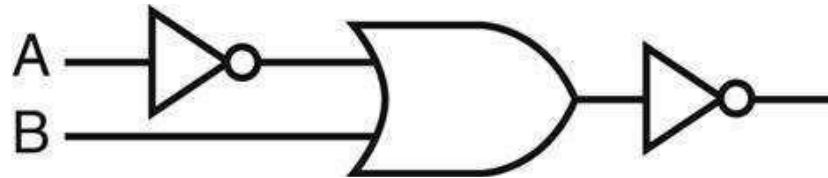
AND



OR



Inverter (NOT)



$$(A^c \cup B)^c = A \cap B^c$$

$$\overline{\bar{A} \text{ OR } B} = A \text{ AND } \bar{B}$$

$$\bar{A} = \text{not } A$$

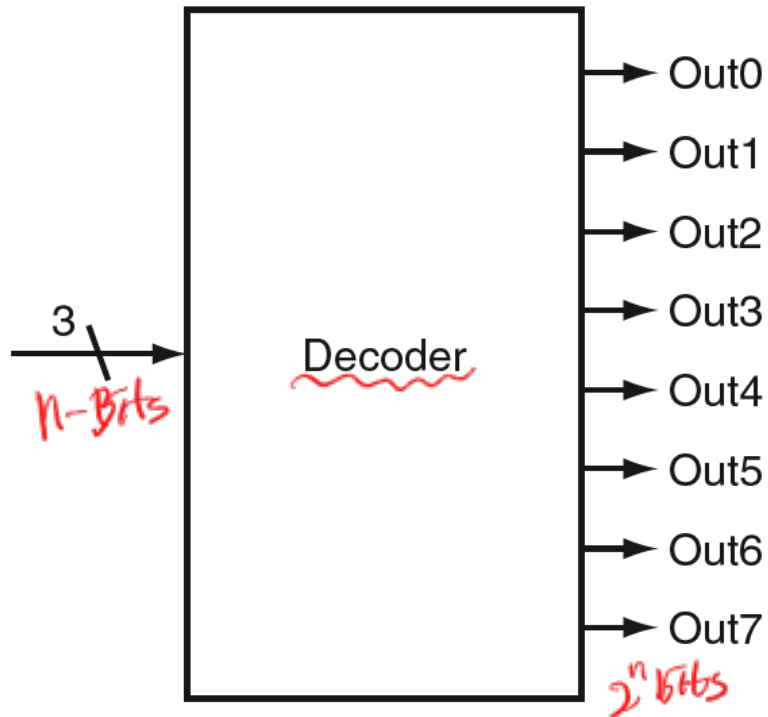
$$\overline{X \text{ and } Y} = \bar{X} \text{ or } \bar{Y}$$

Logic Design Basics: Logic Gates



Logic Design Basics: Decoder

- n -bit input, 2^n -bit output



a. A 3-bit decoder

Inputs			Outputs							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

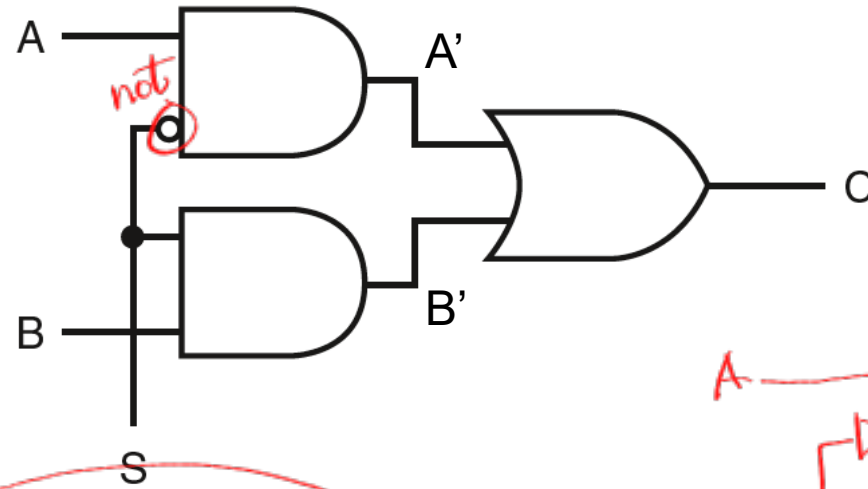
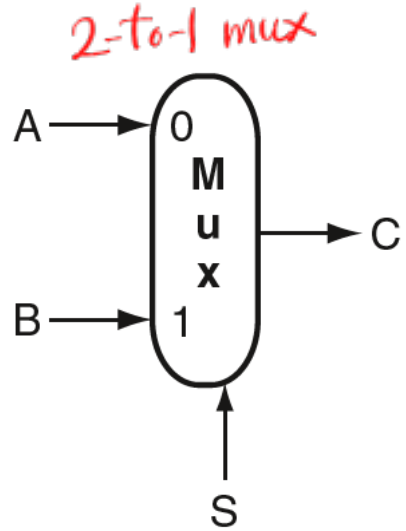
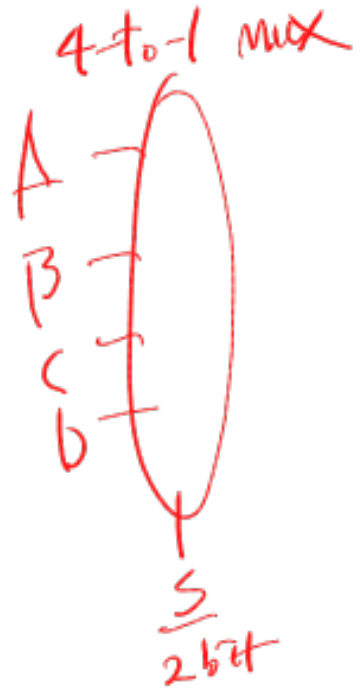
$111_2 = 7$
 $011_2 = 3$

b. The truth table for a 3-bit decoder

1000
0100
0010
0001

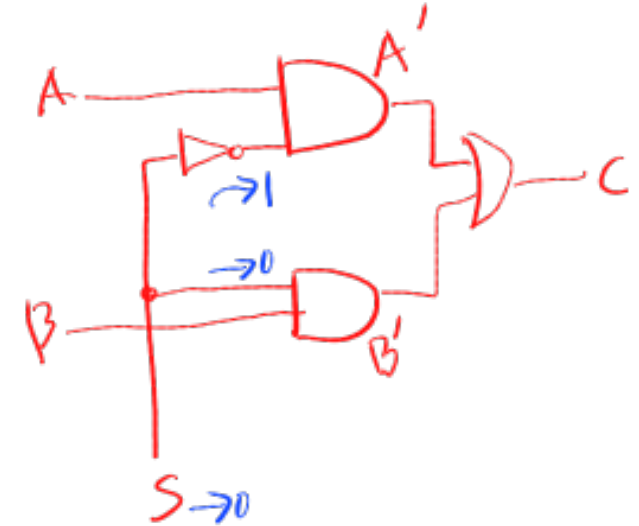
one-hot encoding

Logic Design Basics: Multiplexor (Selector)



S	A	B
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

A'	B'	C
0	0	0
0	0	0
1	0	1
1	0	1
0	0	0
0	1	1
0	0	0
0	1	1



Logic Design Basics: Don't Cares

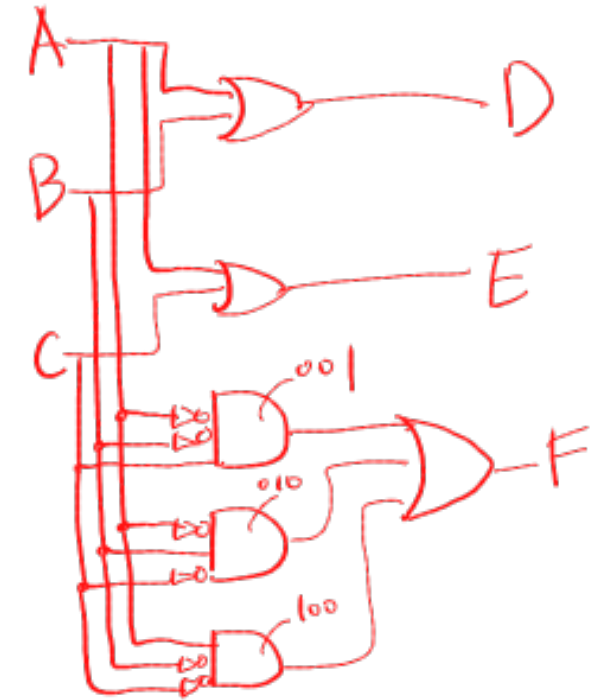
- Either '0' or '1' can be the value
- Represented by an 'X'
- Important for logic optimization → implementing the function with fewer gates

Logic Design Basics: Don't Cares

■ Example:

- ❖ If A or B is true, then output D is true
- ❖ If A or C is true, then output E is true
- ❖ Output F is true if exactly one of the inputs is true, ~~although we don't care about the value of F, whatever D and E are both true.~~

<i>Input</i>			<i>output</i>		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	1	0	1
0	1	1	1	1	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	1	0



Logic Design Basics: Don't Cares

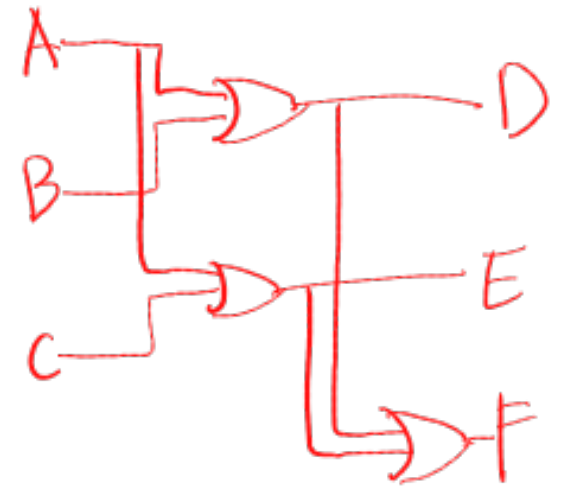
→ 3각화기 응용, 2각화기 응용 같은 경우

■ Example:

- ❖ If A or B is true, then output D is true
- ❖ If A or C is true, then output E is true
- ❖ Output F is true if exactly one of the inputs is true, although we don't care about the value of F, whatever D and E are both true.

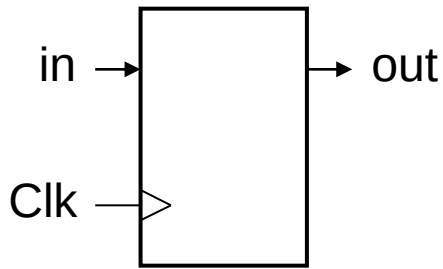
A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

D	E	F
0	0	0
0	1	1
1	0	1
1	1	X
1	1	X
1	1	X
1	1	X
1	1	X

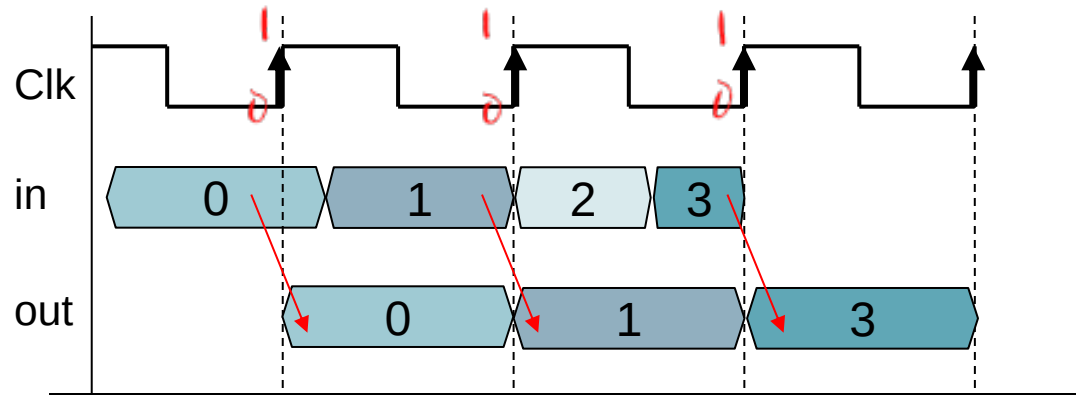


Logic Design Basics: Sequential Elements (State Elements)

- Register: stores data
 - ❖ Uses a clock signal to determine when to update the stored value
 - ❖ Edge-triggered: update when Clk changes from 0 to 1

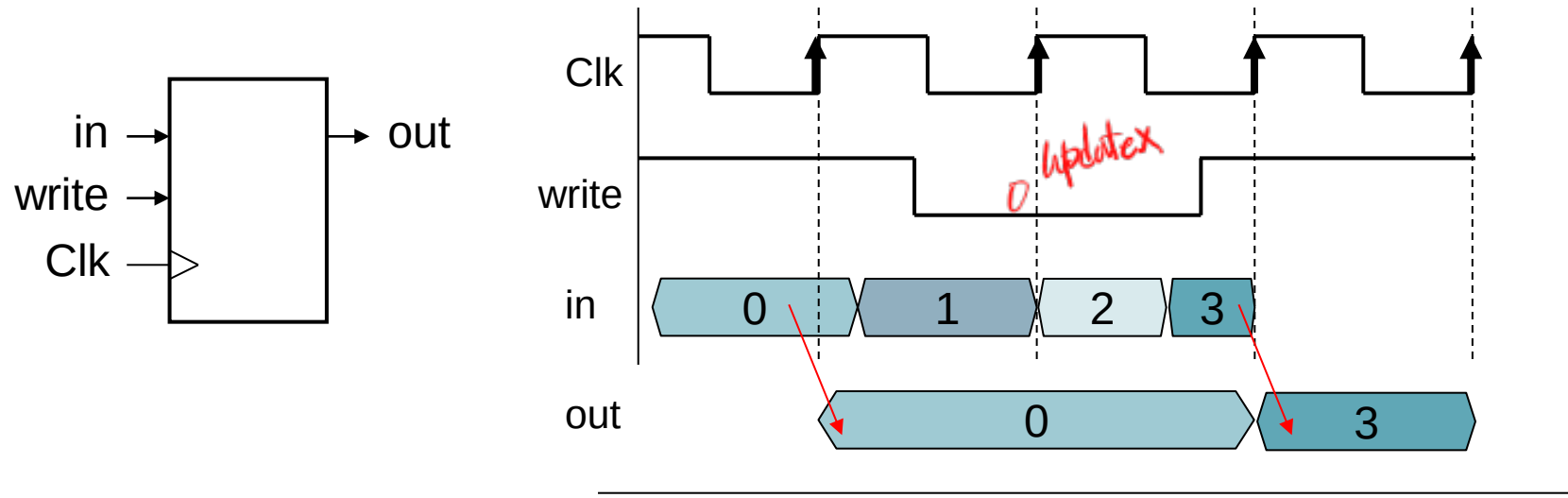


*Edge-triggered Flip-Flop
1-bit data*



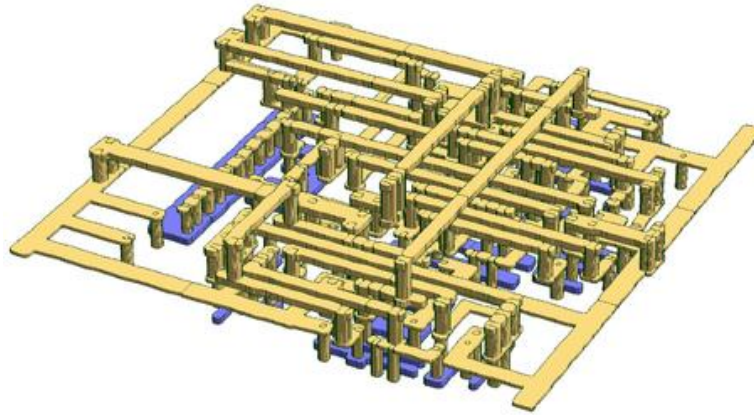
Logic Design Basics: Sequential Elements (State Elements)

- Register with write control
 - ❖ Only updates on clock edge when write control input in 1

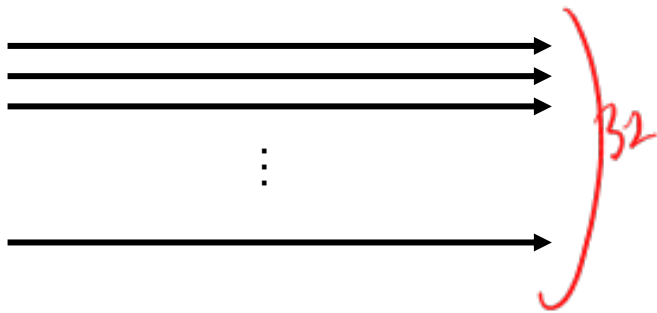


Logic Design Basics: Wires

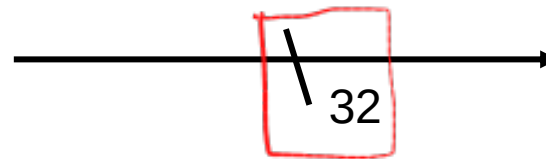
- Logic values (0/1) are delivered through wires



- If you need deliver multiple bits of data, need multiple wires



Simplified representation:

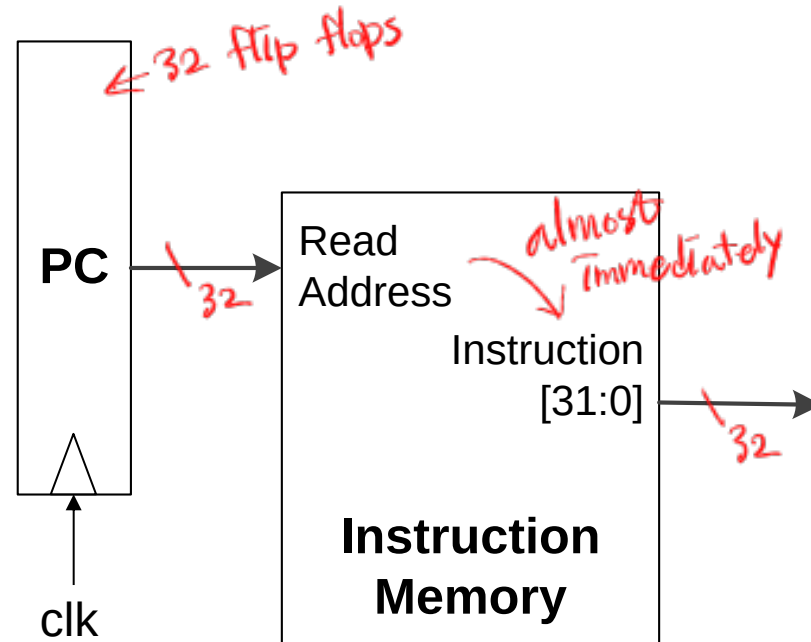


 : n times
n bit data

A red arrow pointing to the right, with a red "n" written below it. To the right of the arrow, the text ": n times" and "n bit data" is written in red.

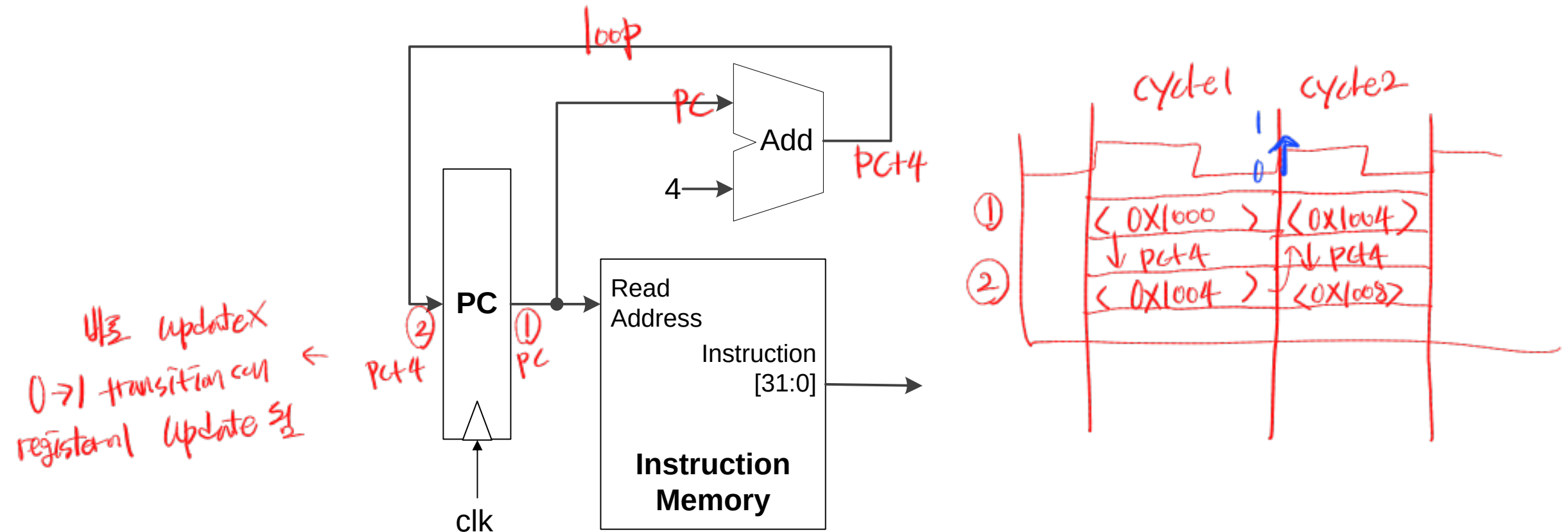
Instruction Fetch

- **PC: a 32-bit register**
 - ❖ Remembers the current instruction address
- **Instruction Memory:**
 - ❖ For now, let's assume it behaves like a combinational logic. i.e., it directly gives us the instruction from the PC address



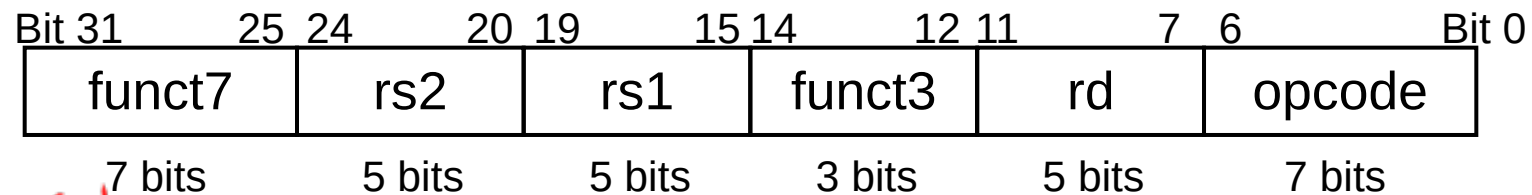
Instruction Fetch

- Advance to the next instruction
 - ❖ Next instruction is at PC + 4
 - ❖ In the next cycle (next clock period), PC should be changed to PC + 4



R-Format Instructions

- Read two register operands
- Perform arithmetic/logic operation
- Write the result into the destination register

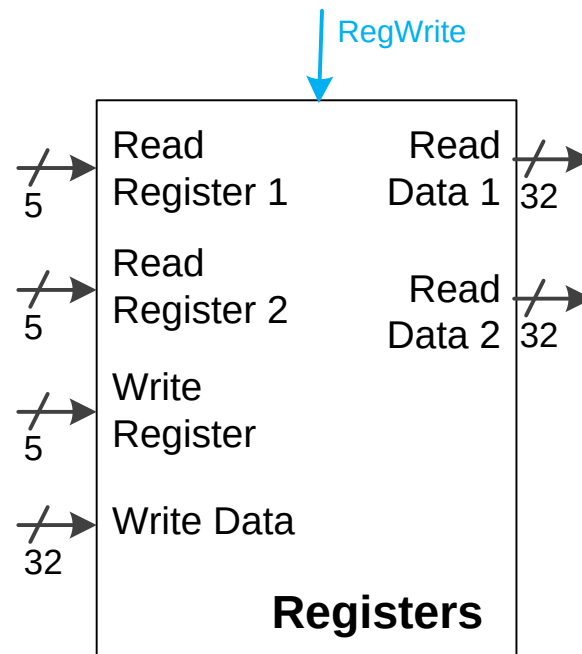


opcode is 5 bits

- add → op: 0110011, funct3: 000 funct7: 0000000
- sub → op: 0110011, funct3: 000 funct7: 0100000
- and → op: 0110011, funct3: 111 funct7: 0000000
- ...

Register File

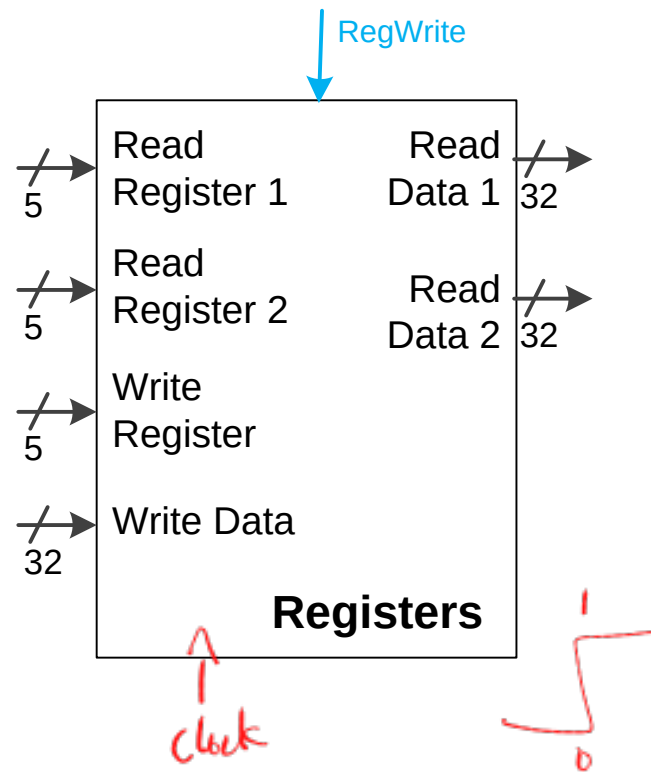
- Read: Behaves like a combinational logic
 - ❖ Read Register 1 / 2: ^{5 bits} Register IDs from the current instruction
 - ❖ Read Data 1 / 2: The current value of the corresponding registers



Register File

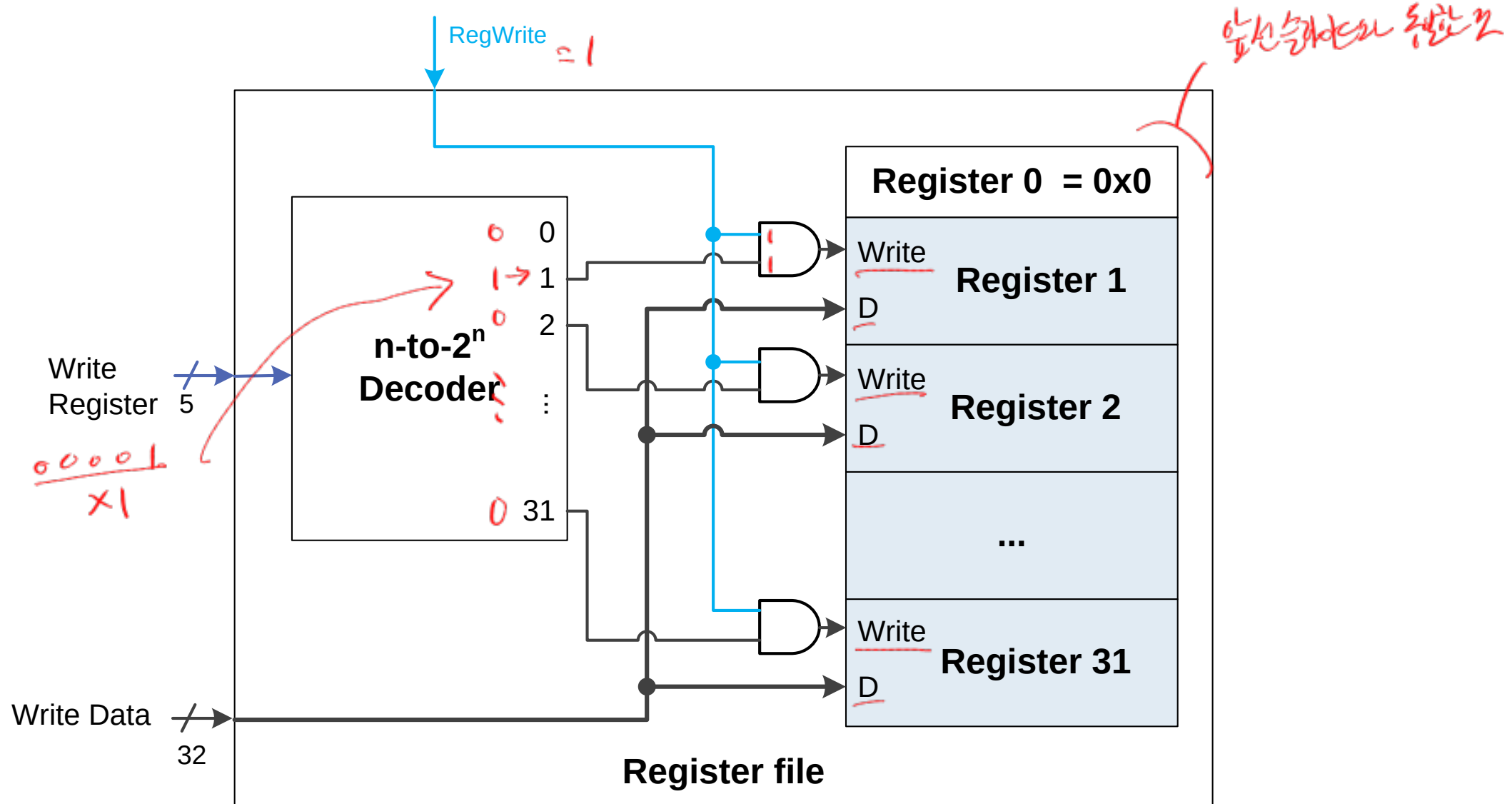
- Write: Behaves like a sequential logic
 - ❖ If RegWrite is "1", writes the value to the target register
 - ❖ Actual write happens when moving to the next cycle

be sw
RegWrite 0 0
Write 1



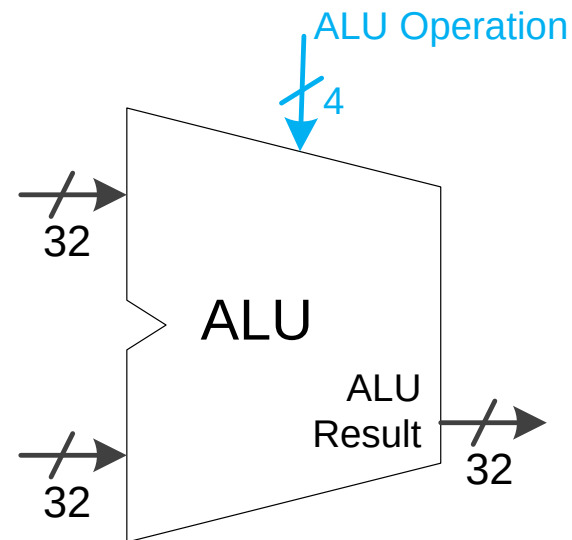


Write Port



Arithmetic Logic Unit (ALU)

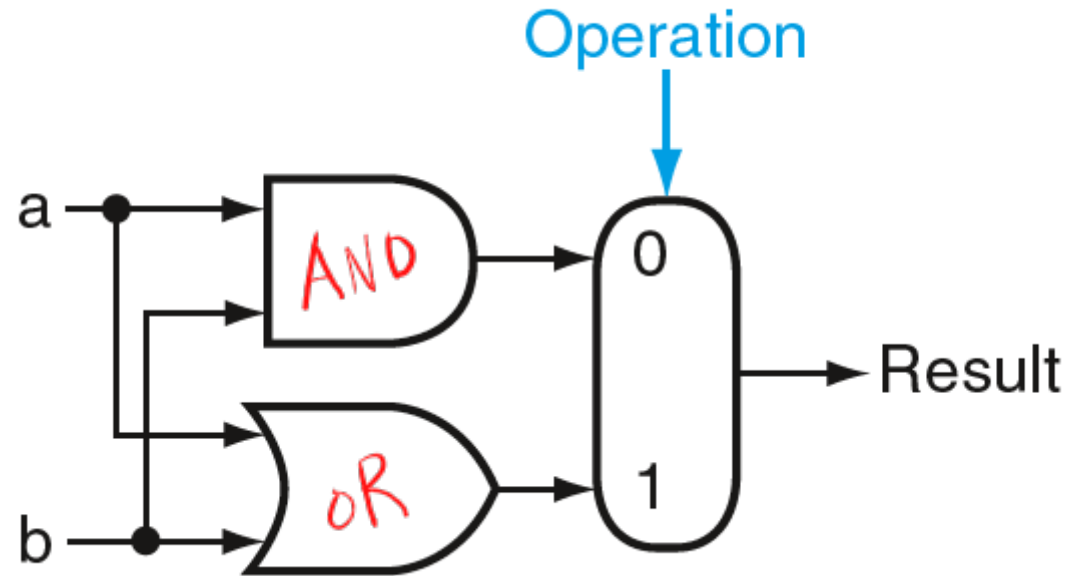
- Multiple functions
 - ❖ Add, subtract, shift, and, or, ...
 - ❖ ALU operation (4-bit input) selects actual function



4 types (Add
Sub
And
OR)

Logic Design Basics: 1-bit ALU

- AND / OR

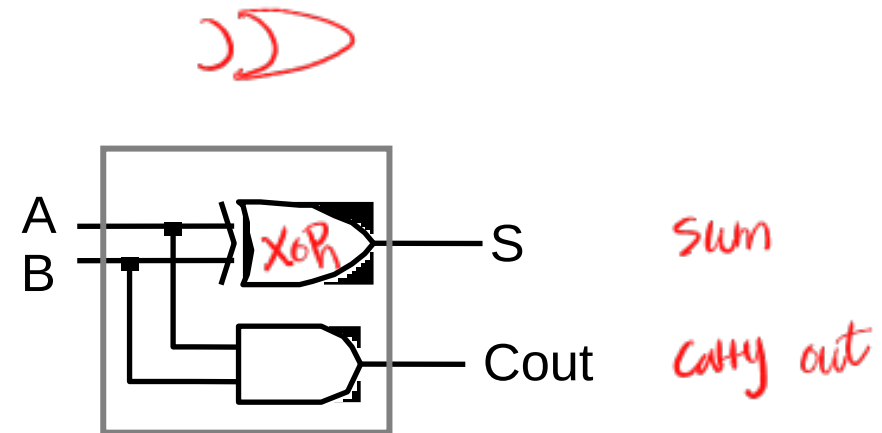


Logic Design Basics: 1-bit Binary Adder

- Inputs: A, B
- Outputs: Sum (S), Carry-out (Cout)

A	B	Cout	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

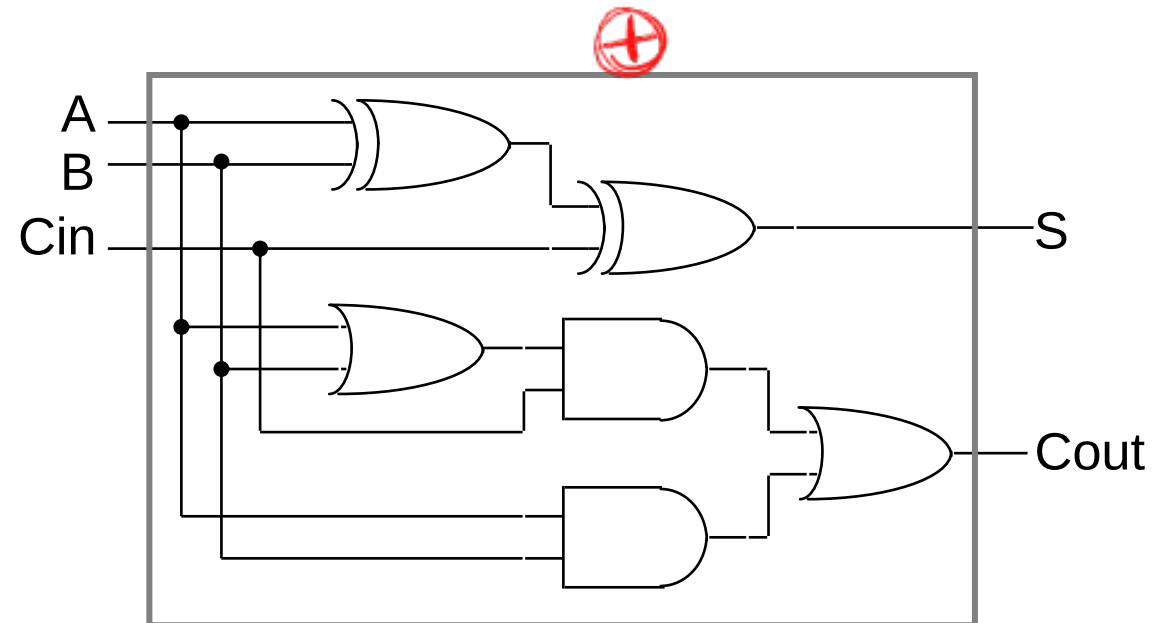
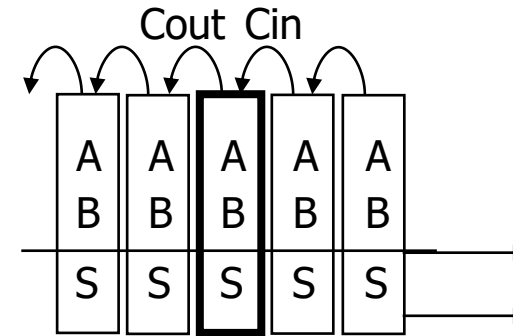
AND *XOR*



Logic Design Basics: 1-bit Binary Full Adder

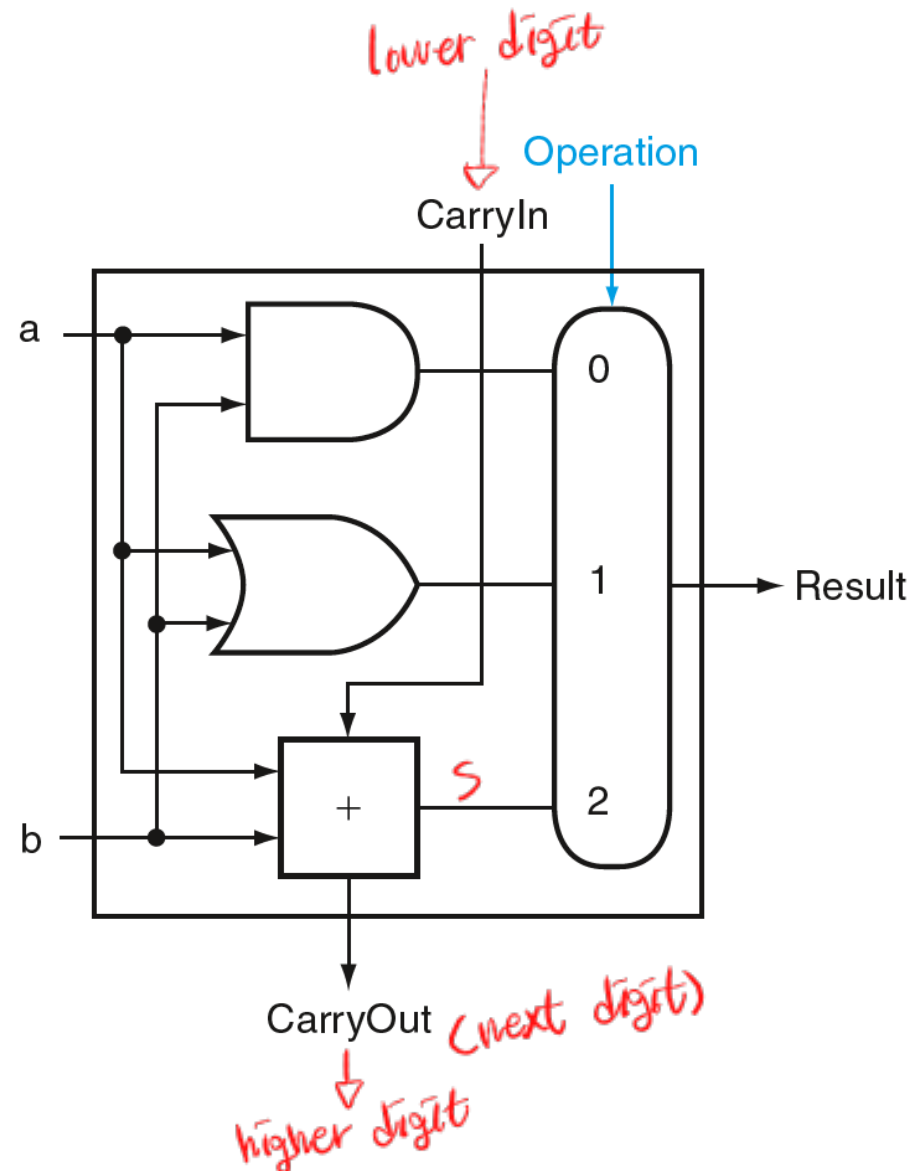
- Inputs: A, B, Carry-in
- Outputs: Sum, Carry-out

A	B	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

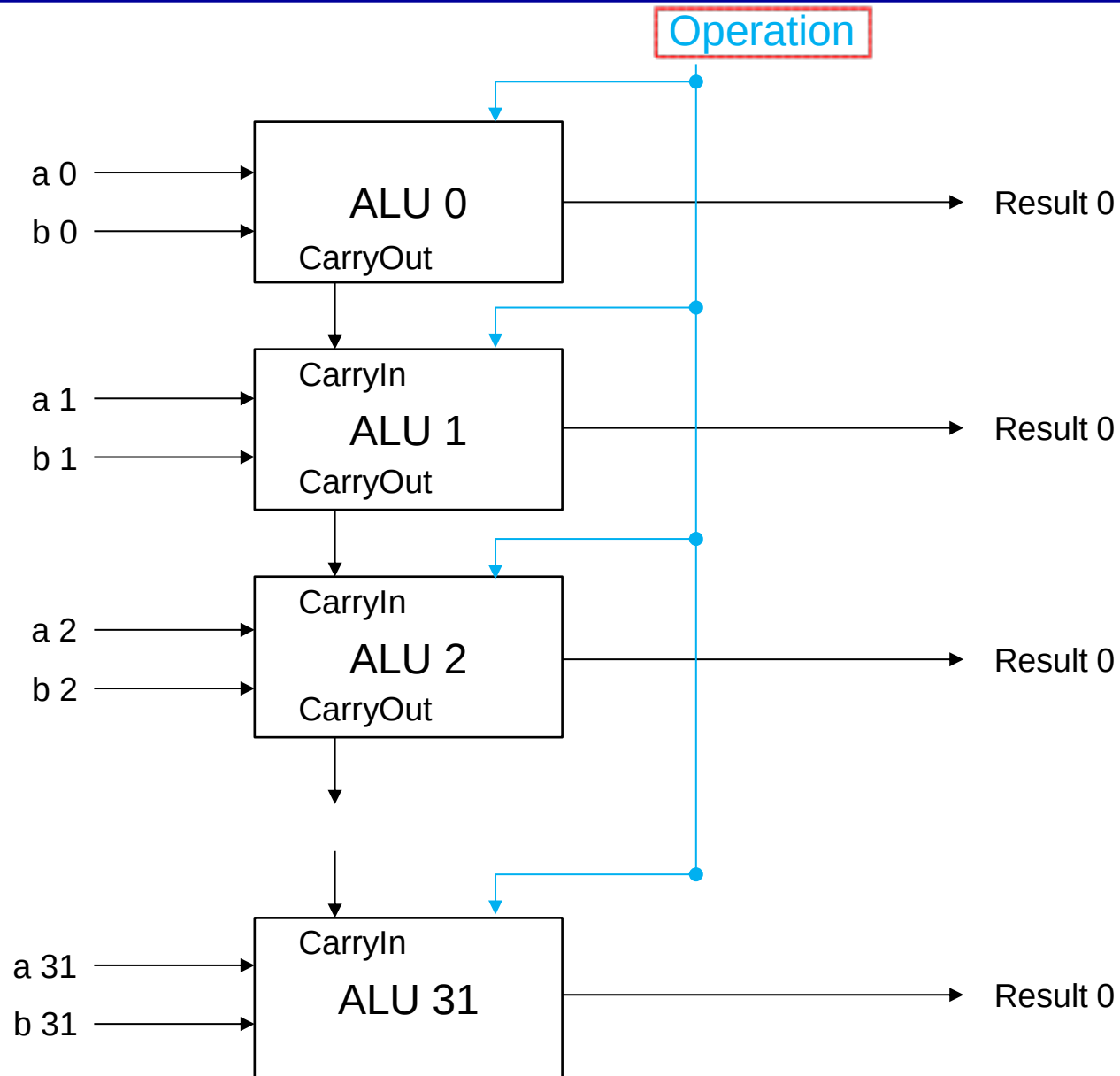


Logic Design Basics: 1-bit ALU

- AND / OR / Addition



Logic Design Basics: 32-bit ALU



ALU in Verilog Example

```
module RISCVALU (ALUctl, A, B, ALUOut, Zero);
  input [3:0] ALUctl;
  input [31:0] A, B;
  output reg [31:0] ALUOut;
  output Zero;
  assign Zero = (ALUOut==0); //Zero is true if ALUOut is 0
  always @(ALUctl, A, B) begin
    case (ALUctl)
      4'b0000: ALUOut <= A & B;
      4'b0001: ALUOut <= A | B;
      4'b0010: ALUOut <= A + B;
      4'b0110: ALUOut <= A - B;
      4'b0111: ALUOut <= A < B ? 1 : 0;
      4'b1100: ALUOut <= ~(A | B); // nor
      default : ALUOut <= 0;
    endcase
  end
endmodule
```

ports

synthesis tool

behavior

Arithmetic Logic Unit (ALU)

■ ALU used for

- ❖ Load/Store: Function = add
- ❖ *beq* Branch: Function = subtract
- ❖ R-type: Function is determined by the funct7 & funct3 fields

Depends on Each CPU Design

ALU control lines (Operation)	ALU Function
0000	AND
0001	OR
0010	add
0110	subtract
...	...
...	...

ALU Control Bits

Opcode	ALUOp	Operation	Funct7	Funct3	ALU function	ALU control
lw	00	load word	XXXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
		sub	0100000	000	subtract	0110
		and	0000000	111	AND	0000
		or	0000000	110	OR	0001

Handwritten notes:

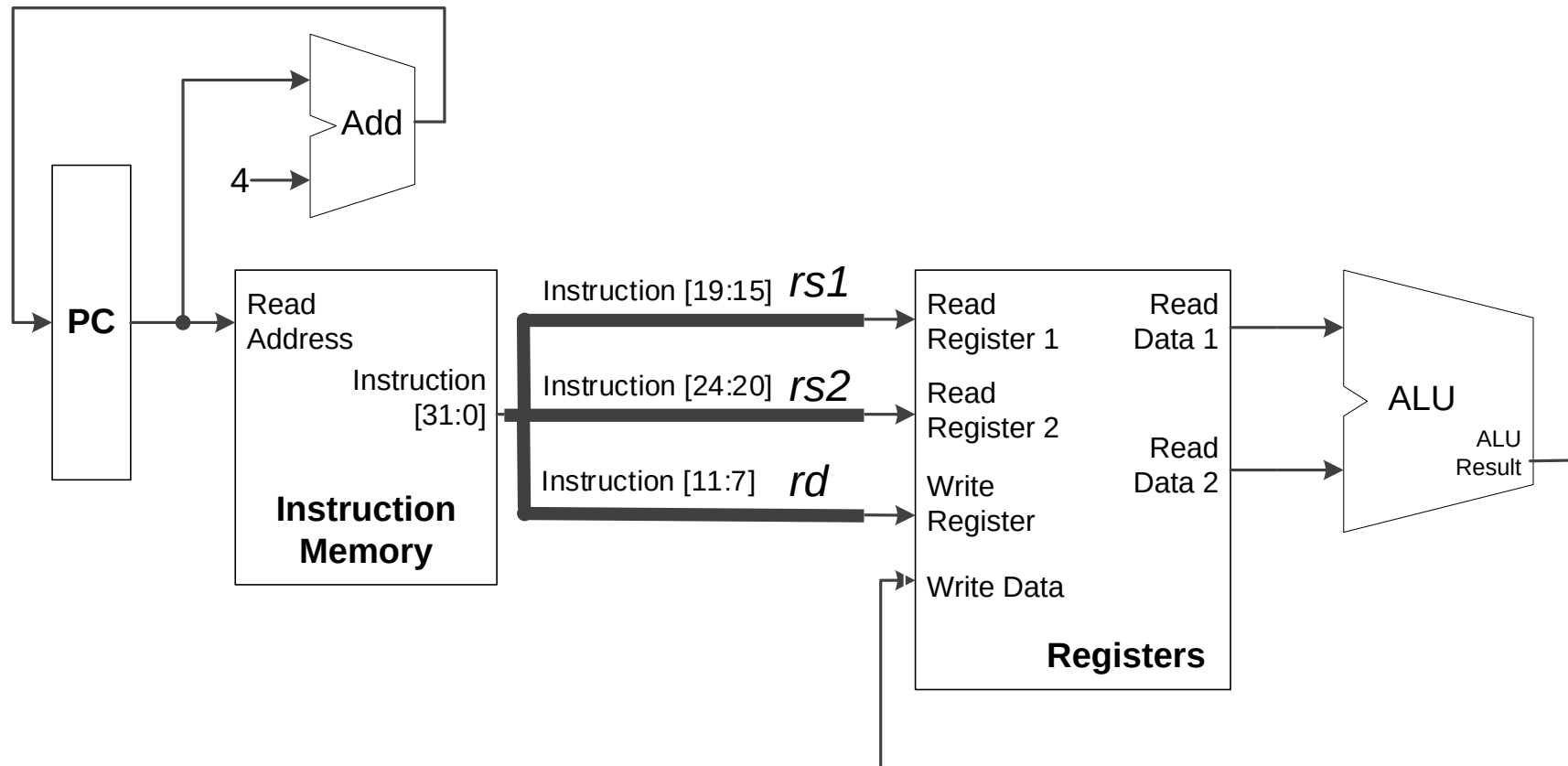
- 2 bit signal (pointing to ALUOp)
- perform Add, regardless of funct 7 & funct 3 (pointing to ALU function)
- " Sub, " (pointing to ALU function)
- check funct 7 & funct 3 for determining ALU control (pointing to ALU control)

ISA

기타의 필요성

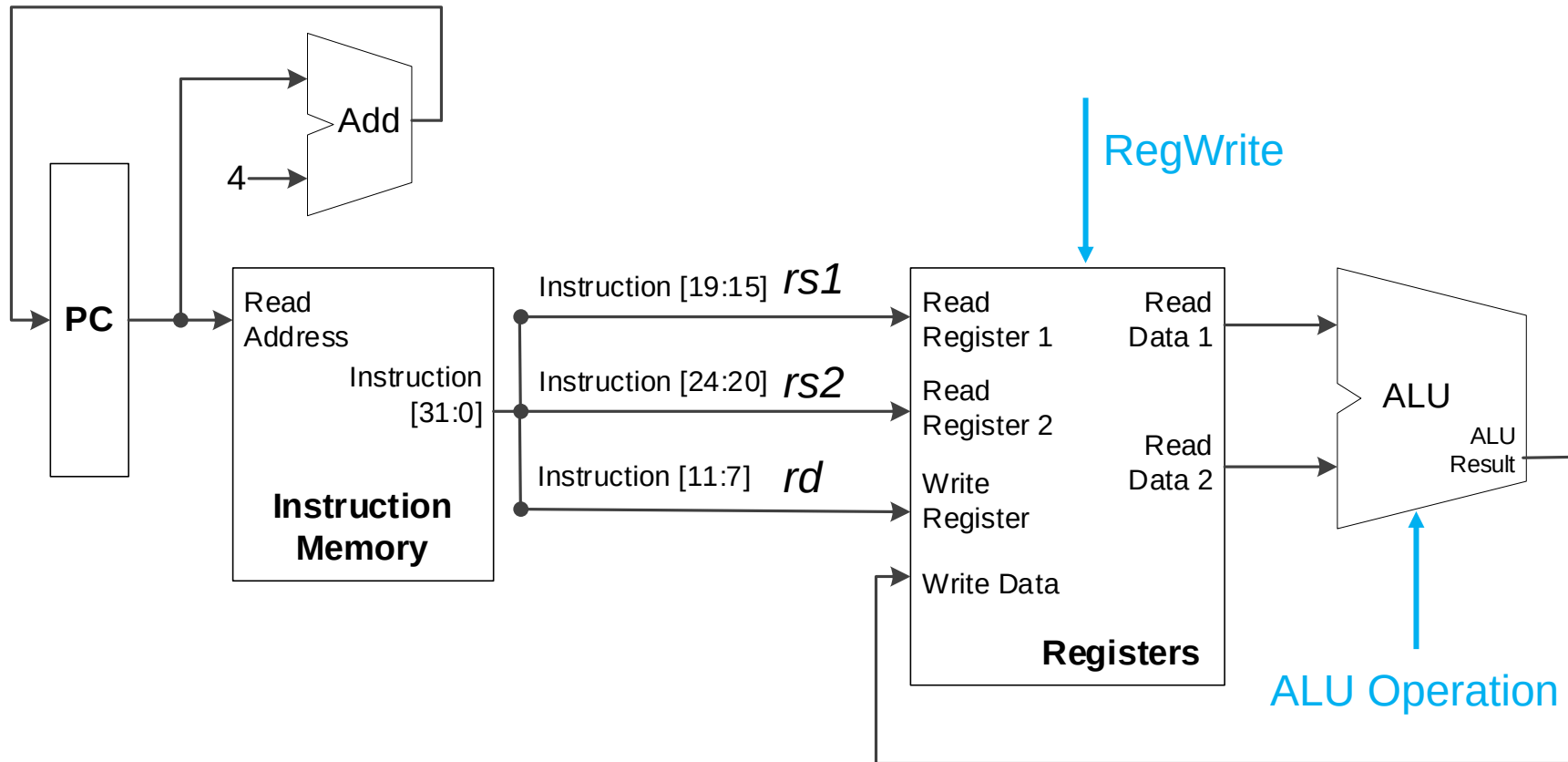
CPU for R-type Instructions

■ Datapath



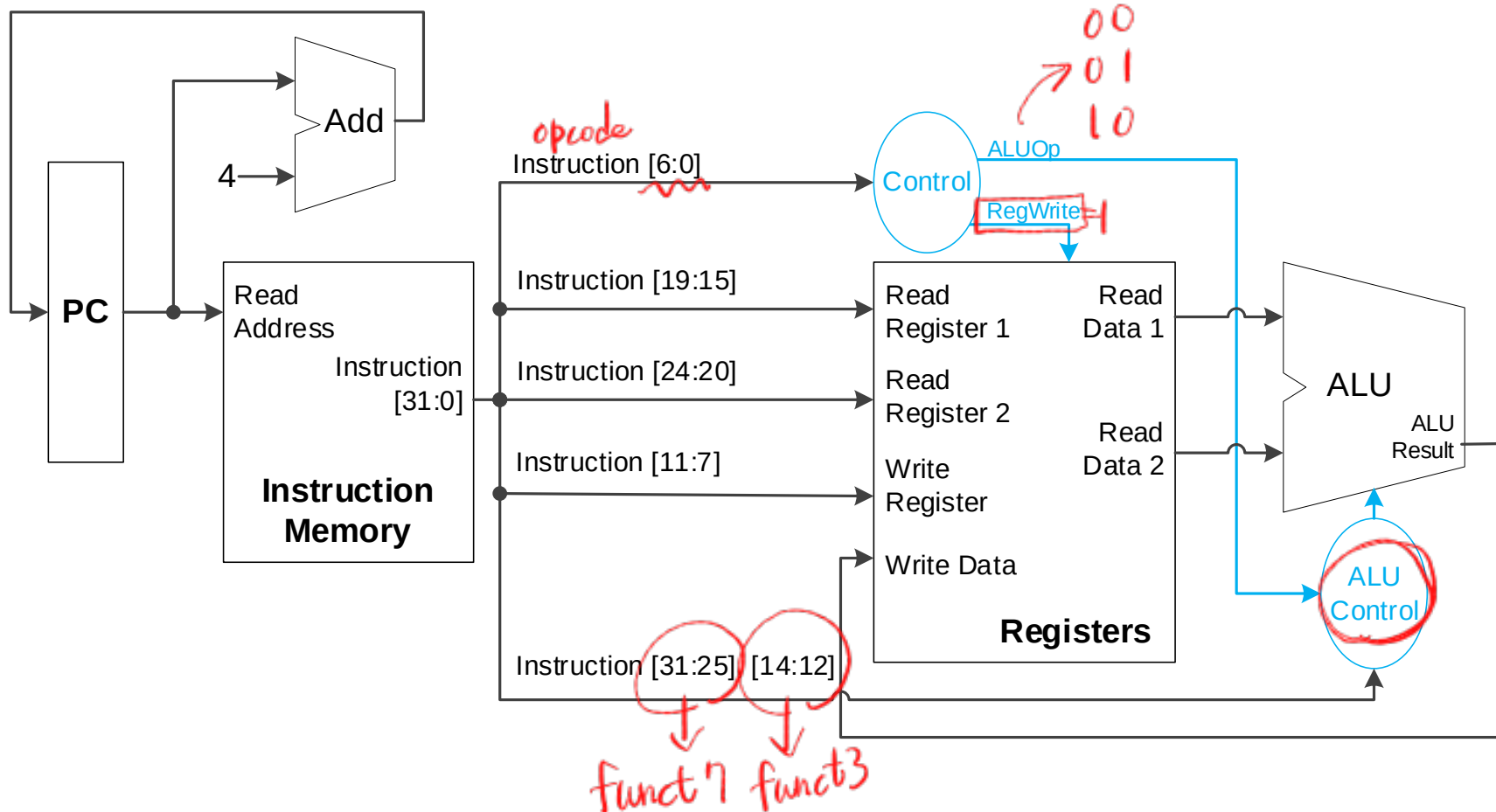
CPU for R-type Instructions

■ Datapath

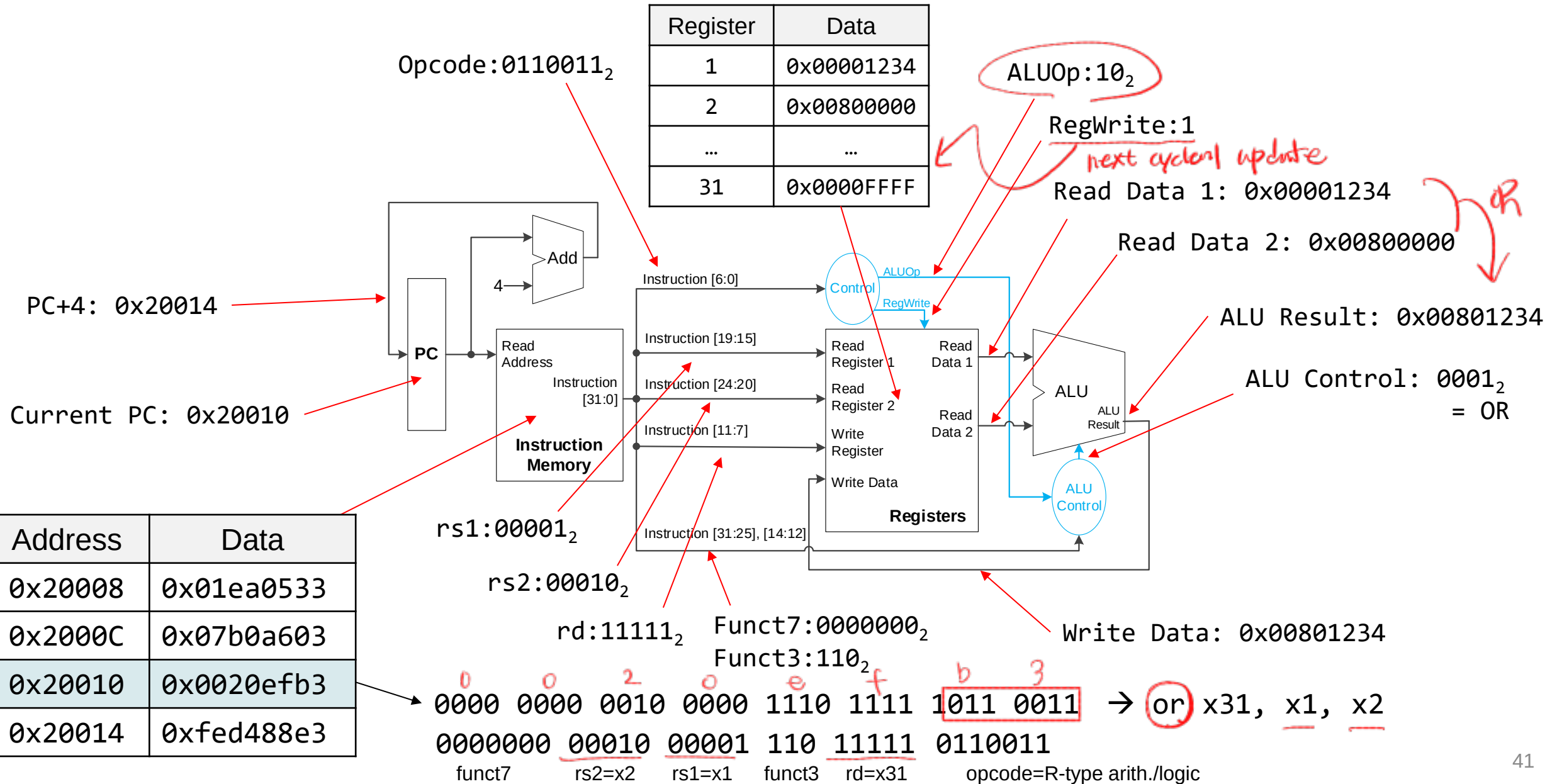


CPU for R-type Instructions

■ Datapath + Control signals

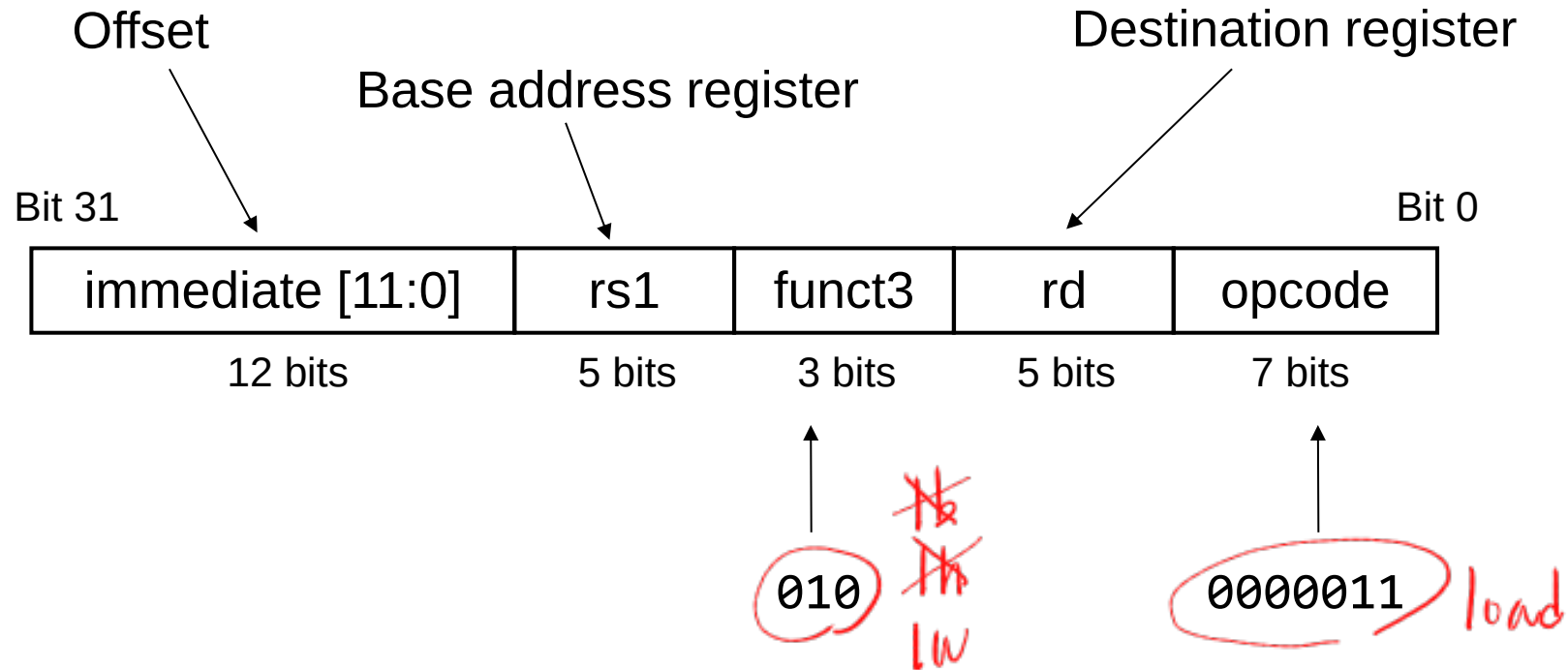


R-Type Execution Example



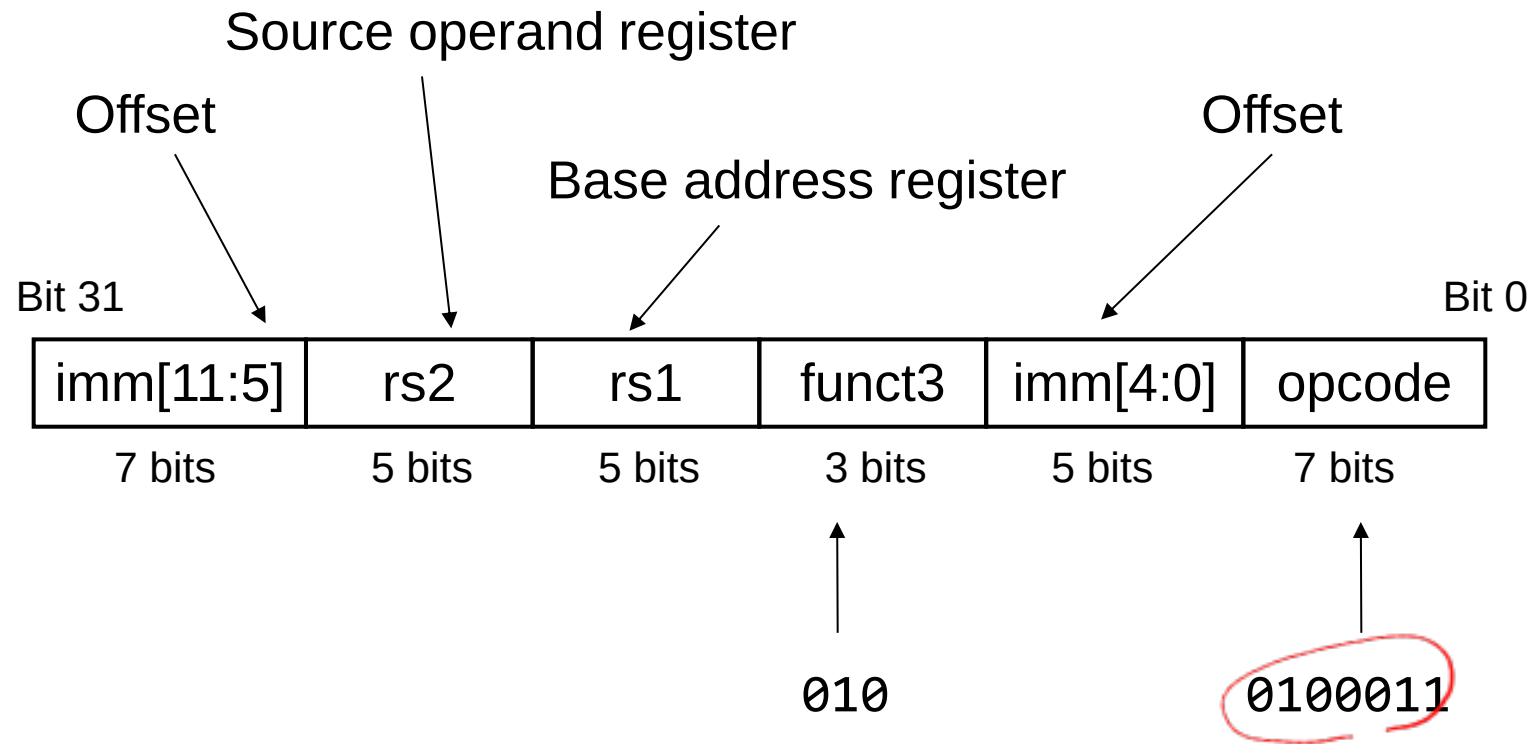
Load Instruction

- Address: Base address register + Offset



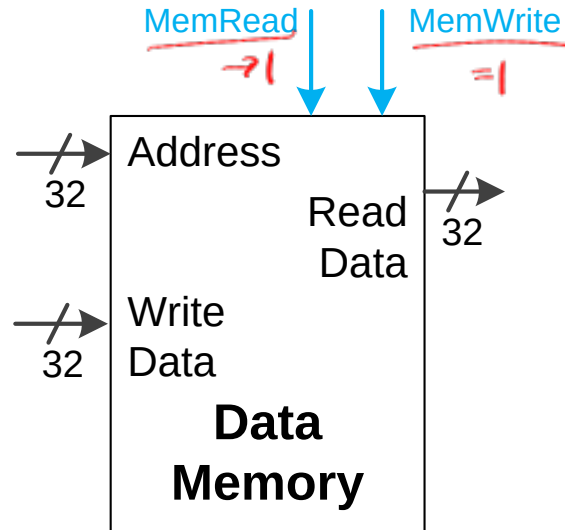
Store Instruction

- Address: Base address register + Offset



Data Memory

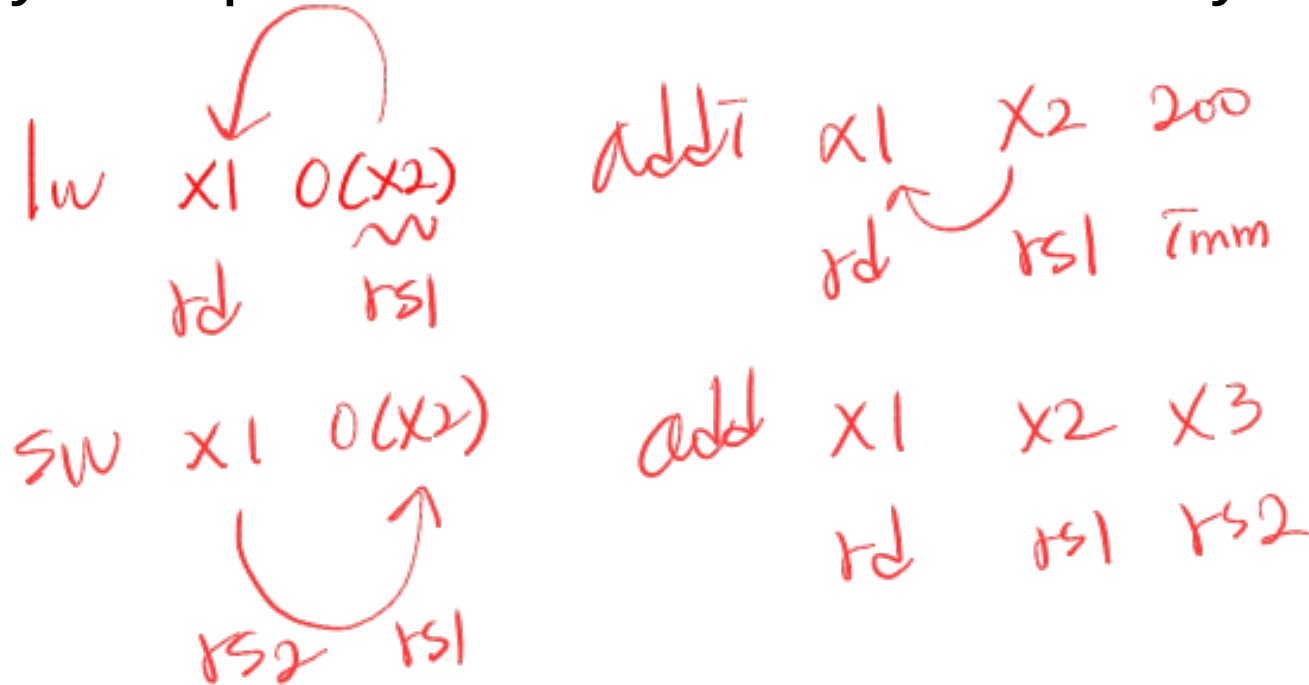
- Read: Assume it behaves like a combinational logic
 - ❖ If **MemRead** is “1”, gives the current memory value from the address
- Write: Behaves like a sequential logic
 - ❖ If **MemWrite** is “1”, writes the data to the address



- Accessing main memory takes a long time
 - ❖ Typically, hundreds of cycles
 - ❖ Cache memory reduces the access time
- For now, we assume memory access does not add long latencies
 - ❖ i.e., assume we always have cache hit

Why no RegRead but MemRead ?

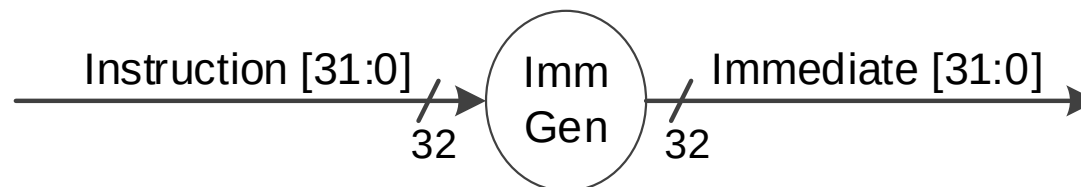
- Reading register is cheap. There is no great harm to read it always.
- Reading memory is expensive. Avoid if not absolutely needed.



Immediate Generation Unit (ImmGen)

- RISC-V instructions have immediate fields in various bit places
 - ❖ Can identify the format from the opcode
 - ❖ Sign-extend

I format	immediate [11:0]	rs1	funct3	rd	opcode
S format	imm[11:5]	rs2	rs1	funct3	imm[4:0] opcode
SB format	12 imm[10:5]	rs2	rs1	funct3	imm[4:1] 11opcode
UJ format	20 imm[10:1]	11 imm[19:12]	rd	opcode	
U format	imm[31:12]			rd	opcode



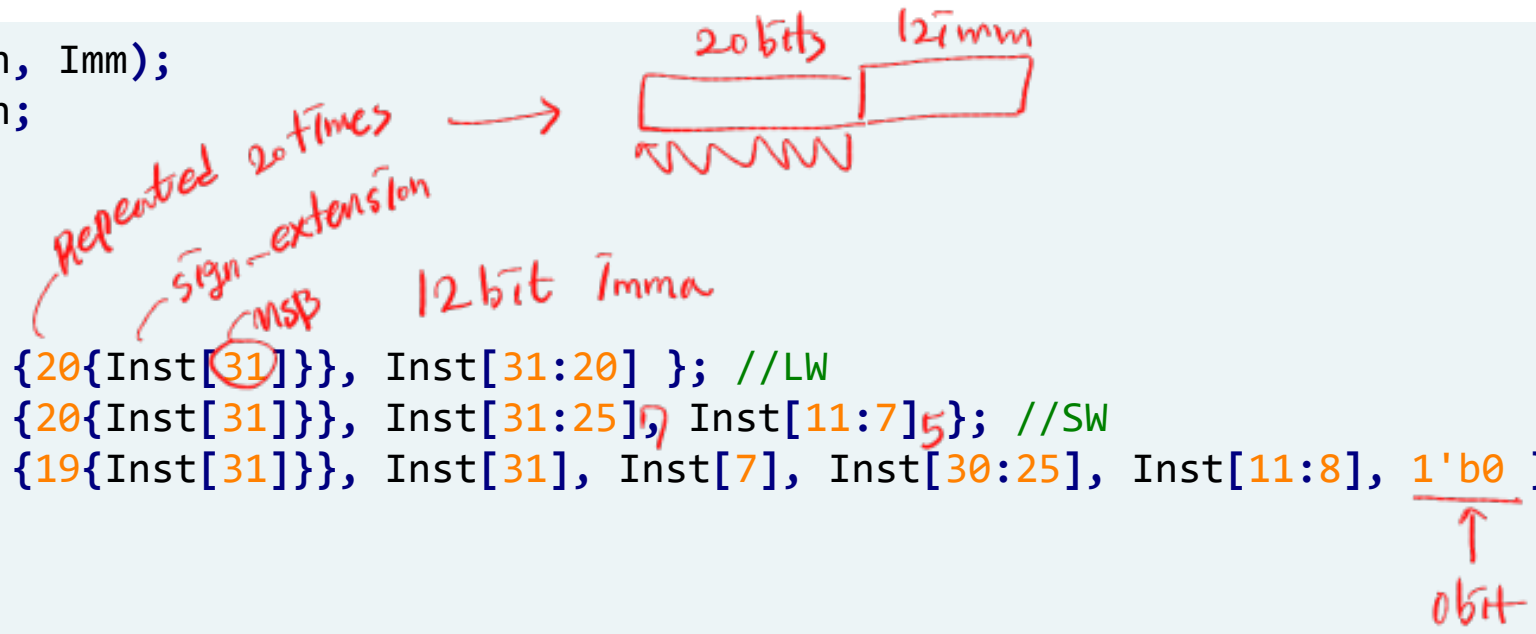
Immediate Generation Unit (ImmGen) in Verilog Example

```
module ImmGen (Instruction, Imm);
  input [31:0] Instruction;
  output reg [31:0] Imm;

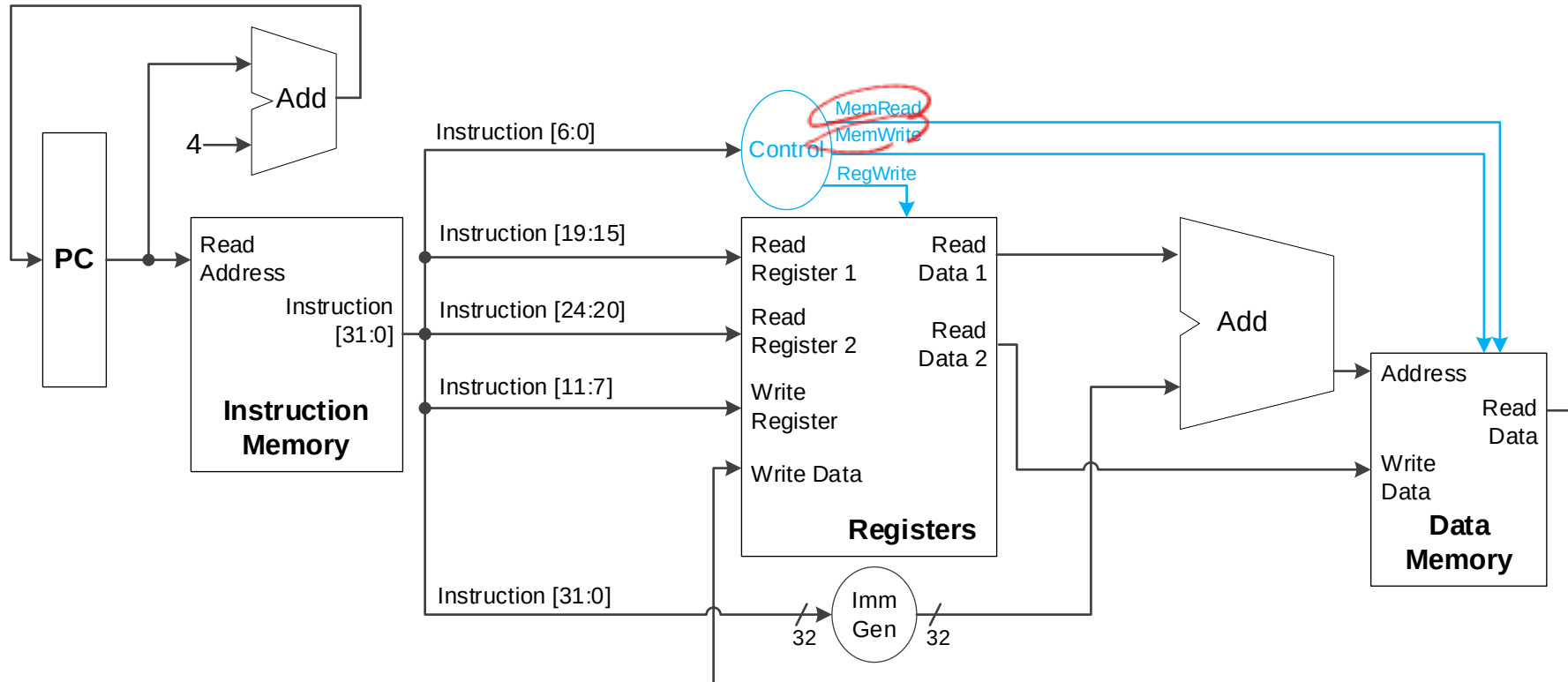
  always @(Inst) begin
    case (Inst[6:0])
      7'b0000011: Imm = { {20{Inst[31]}}, Inst[31:20] }; //LW
      7'b0100011: Imm = { {20{Inst[31]}}, Inst[31:25], Inst[11:7] }; //SW
      7'b1100011: Imm = { {19{Inst[31]}}, Inst[31], Inst[7], Inst[30:25], Inst[11:8], 1'b0 }; //BEQ
      default : Imm = 0;
    endcase
  end
endmodule
```

Handwritten annotations:

- Repeated 20 times* (with an arrow pointing to the 20-bit field in the LW case)
- sign-extension* (with an arrow pointing to the 20-bit field in the LW case)
- msb* (with a circle around the 31st bit of the LW case)
- 12 bit Imma* (with an arrow pointing to the 12-bit field in the SW case)
- 0 bit* (with an arrow pointing to the 1'b0 field in the BEQ case)



CPU for Load/Store



	MemRead	MemWrite	RegWrite
Load	1	0	1
Store	0	1	0

Store Execution Example

